

CO - 5 ASSESSMENT TOOL -2

Name : Lanka Kavya
Registration Number : 192472298

Subject Code : CSA0619
Subject Name : DAA

Q17 : Develop a pseudocode to determine satisfiability of a 3-SAT Boolean expression using Backtracking. Clearly identify inputs (Boolean clauses) and output (true/false). Design an algorithm that assigns truth values to variables systematically. Use control structures to evaluate clauses dynamically. Apply pruning to eliminate unsatisfied assignments early. Ensure the pseudocode is structured and readable. Handle all possible cases.

Sol :

Source code :

```
def check(clauses, assign):  
    for clause in clauses:  
        if not any((x > 0 and assign[x]) or  
                   (x < 0 and not assign[-x]) for x in clause):  
            return False  
    return True
```

```
def solve(clauses, vars, assign, i=0):  
    if i == len(vars):  
        return check(clauses, assign)
```

```
    x = vars[i]
```

```
    assign[x] = True  
    if solve(clauses, vars, assign, i + 1):  
        return True
```

```
    assign[x] = False  
    if solve(clauses, vars, assign, i + 1):  
        return True
```

```
    del assign[x]  
    return False
```

```
clauses = [[1, 2, -3], [-1, 2, 3], [1, -2, 3]]  
variables = [1, 2, 3]
```

```
assignment = {}
```

```
if solve(clauses, variables, assignment):  
    print("SATISFIABLE")  
    print(assignment)  
else:  
    print("NOT SATISFIABLE")
```

Aim

To determine whether a 3-SAT Boolean expression is satisfiable using the backtracking technique.

Objective

- Assign True or False to each variable.
- Check whether all clauses are satisfied.
- Backtrack when an assignment fails.
- Prune invalid assignments early.

Input

A Boolean expression containing clauses with three literals.

Example:

```
(x1 OR x2 OR NOT x3)  
AND  
(NOT x1 OR x2 OR x3)  
AND  
(x1 OR NOT x2 OR x3)
```

Output

- SATISFIABLE if a valid assignment exists.
- NOT SATISFIABLE otherwise.

Algorithm

1. Select the next unassigned variable.
2. Assign True.
3. Recursively solve the remaining variables.
4. If it fails, assign False.
5. If both values fail, backtrack.
6. If all variables are assigned and every clause is satisfied, return True.

Pruning

If a clause becomes false during the search, that assignment is rejected immediately. This avoids exploring unnecessary branches.

Complexity

For n variables, the worst-case time complexity is:

$O(2^n)$

Space complexity:

$O(n)$

Result

The 3-SAT problem was successfully solved using **backtracking**. The algorithm systematically tries truth assignments and eliminates invalid assignments through pruning.

```
# Try True
assignment[variable] = True

if backtrack(clauses, variables, assignment, index + 1):
    return True

# Try False
assignment[variable] = False

if backtrack(clauses, variables, assignment, index + 1):
    return True

# Backtrack
del assignment[variable]

return False

def three_sat(clauses, variables):
    assignment = {}

    result = backtrack(clauses, variables, assignment, 0)

    return result, assignment

# Example 3-SAT expression:
# (x1 OR x2 OR NOT x3)
# AND
# (NOT x1 OR x2 OR x4)
# AND
# (x1 OR NOT x2 OR x4)

clauses = [
    [1, 2, -3],
    [-1, 2, 4],
    [1, -2, 4]
]

variables = [1, 2, 3, 4]

result, assignment = three_sat(clauses, variables)

print("SATISFIABLE")
print("Assignment:", assignment)
```



Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10 AMD64) on win32
Enter "help" below or click "Help" above for more in:

```
>>>
===== RESTART: C:/Users/Kavya sri/OneDrive/Document:
SATISFIABLE
Assignment: {1: True, 2: True, 3: True, 4: True}
>>>
===== RESTART: C:/Users/Kavya sri/OneDrive/Document:
SATISFIABLE
Assignment: {1: True, 2: True, 3: True, 4: True}
>>>
```